CERTIFIED AGENTIC AI ENGINEER

2026 EDITION

One-Year Course Outline | 4 Modules × 3 Months | 312 Total Instruction Hours

Course Philosophy

Agentic AI has crossed from experimentation into execution. By 2026, Gartner predicts 40% of enterprise applications will embed AI agents, and IDC expects AI agents to be present in 80% of workplace tools. This curriculum is built to produce engineers who can design, build, and ship production-grade agentic systems — not just demos.

Schedule: 3 days/week × 2 hours/class = 6 hours/week × 52 weeks ≈ 312 total instruction hours

Mandatory 2026 Technology Stack

Based on current industry research, the following technologies are non-negotiable for agentic AI engineers in 2026:

Category	Technologies
Language	Markdown, Python, Prompt & Context Engineering
Agent Frameworks	OpenAI Agents SDK, Google ADK, LangChain v1, LangGraph *(One track selected before course launch)*
Agent Protocols	MCP (Model Context Protocol), A2A (Agent-to-Agent)
Autonomous Agents	OpenClaw, NanoClaw
Memory	mem0 or LangMem *(One selected before course launch)*
RAG & Vector DB	Qdrant or Pinecone *(One selected before course launch)*
Observability	LangSmith, Langfuse, Arize AI
Infrastructure	Docker, Microservices
Data Layer	SQLModel, Alembic, PostgreSQL / Neon DB (Serverless Postgres)
LLM Providers	OpenAI, Anthropic, Google, xAI (Grok), DeepSeek, Meta (LLaMA), Groq, Open-source via Ollama
Security	OAuth 2.1, NANDA identity protocol
No-Code Automation	n8n (visual workflow builder for agents)
AI Coding CLI	Claude Code (Anthropic's agentic coding CLI), Spectkit (Spec Driven Development)

Course Structure Overview

Module	Title	Duration	Focus
1	Intro to AI (Predictive / Generative & Agentic), n8n (Low-Code Agents), Python (Basics → Advanced), Prompt / Context Engineering, Spec Driven Development, Claude Code (Skills, Sub-agents, Plugins)	3 Months	Foundations
2	OpenAI Agents SDK / Google ADK / LangChain v1 + LangGraph (one track), Short-Term Memory, RAG (with Qdrant or Pinecone), Agentic Design Patterns, Multi-Agent Systems, Long-Term Memory (mem0 or LangMem)	3 Months	Intelligence Layer
3	FastAPI, Neon DB, SQLModel, Alembic, Protocol Foundations (HTTP, REST, JSON-RPC), Custom MCP Servers, A2A Protocol, Agent Evaluation & Testing, Tracing & Observability, Fully Autonomous Agents (OpenClaw, NanoClaw), End-to-End Project	3 Months	Collaboration Layer
4	Docker, Microservices, Dapr, DACA Pattern, Production Infrastructure, Ethics / Safety / Governance, Capstone Project	3 Months	Production & Scale

MODULE 1 — Foundations: AI Intro, n8n, Python, Prompt Engineering & Claude Code

Duration	3 Months (Weeks 1–12)
Total Hours	~78 hours
Prerequisite	Basic Python knowledge

Module Goal

Understand the AI landscape (predictive, generative, and agentic), build low-code AI agent systems with n8n, strengthen Python skills from basics to advanced for agentic development, master prompt and context engineering, learn AI-powered coding with Claude Code, and deliver a first coded agent project.

Week 1–3 | Introduction to AI & n8n (Low-Code AI Agents)

Week 1: Introduction to AI Paradigms

- What is AI? History and evolution
- Predictive AI: classification, regression, and traditional ML
- Generative AI: text, image, code generation with LLMs
- Agentic AI: autonomous, tool-using, multi-step reasoning systems
- How LLMs work: tokenization basics, context windows, API calling patterns
- LLM provider overview: OpenAI, Anthropic, Google, xAI, DeepSeek, Meta, Groq, Ollama

Week 2: n8n Fundamentals

- What n8n is: an open-source, self-hostable visual workflow automation tool
- n8n vs. Zapier vs. Make — why n8n is the choice for agentic AI engineers
- Installing n8n locally and via Docker
- Core concepts: workflows, nodes, triggers, credentials
- The AI Agent node in n8n — connecting LLMs (OpenAI, Anthropic Claude) as workflow brains

Week 3: Building AI Agent Workflows with n8n

- Building a complete AI agent workflow: input → LLM reasoning → tool calls → output
- Sub-workflows as modular agent components
- Email triage agent: reads inbox, classifies, drafts replies, escalates
- Scheduled research agent: daily brief generation on a topic
- Webhook-triggered agents: responding to external events automatically

Week 4–7 | Python for Agentic AI (Basics → Advanced)

Week 4: Python Basics I

- Data types, variables, and operators
- Control flow: if/else, loops, comprehensions
- Functions, scope, and closures
- Modules and imports

Week 5: Python Basics II

- Object-Oriented Programming: classes, inheritance, polymorphism
- File handling and I/O
- Error handling: try/except patterns
- Working with JSON and APIs

Week 6: Advanced Python I

- Decorators and context managers
- Generators and iterators
- Type hints and Pydantic v2 for data validation
- Python packaging with uv (the modern replacement for pip)

Week 7: Advanced Python II

- Asynchronous Python: asyncio, async/await
- Async patterns for agent development
- Virtual environments and pyproject.toml
- Git, GitHub, and collaborative workflows
- Development environment setup

Week 8–9 | Prompt & Context Engineering + Spec Driven Development

Week 8: Prompt Engineering

- Markdown fundamentals: syntax, formatting, headers, lists, links, code blocks
- Markdown for prompt writing and documentation
- Zero-shot, one-shot, and few-shot prompting
- Chain-of-thought (CoT) and tree-of-thought prompting
- Role-playing, persona, and constraint-setting
- ReAct pattern (Reason + Act) for tool-using agents
- Prompt chaining and decomposition
- System prompt design for agentic behavior
- Guardrail prompts and safety constraints
- Structured output prompting with JSON schemas

Week 9: Context Engineering + Spec Driven Development

- What context engineering is: designing the full information environment an LLM operates in
- Context window management strategies
- Dynamic context injection and retrieval
- Context compression and summarization techniques
- Spec Driven Development: writing specifications before code

- Using Spectkit for AI-assisted spec-driven workflows
- Manual prompt evaluation frameworks
- A/B testing prompts systematically
- Introduction to LangSmith for tracing and evaluation

Week 10–11 | Claude Code

Week 10: Claude Code Fundamentals

- What Claude Code is: Anthropic's agentic CLI that reads, writes, and runs code autonomously
- Installing and authenticating Claude Code
- Core workflows: explaining codebases, writing features, fixing bugs, running tests via natural language
- Using Claude Code for agentic project scaffolding and refactoring
- From Spectkit specs to implementation with Claude Code

Week 11: Claude Code Advanced — Skills, Sub-agents & Plugins

- Claude Skills: what they are, how to create and use them
- Claude Sub-agents: delegating tasks to specialized sub-agents
- Claude Plugins: extending Claude Code with third-party integrations
- Best practices: when to trust Claude Code vs. review manually
- Integrating Claude Code into daily development workflow alongside Git

Week 12 | First Agent Build & Module 1 Project

First Coded Agent

- Introduction to building agents with code using the chosen SDK
- Building a simple agent with tool integration
- Agent instructions, personas, and tool configuration

Module 1 Project

Build a working AI agent system — both a low-code n8n workflow and a coded agent — demonstrating prompt engineering skills from Weeks 8–9.

Module 1 Assessment

- Quiz 1: AI Fundamentals & Python (MCQ, 1 hour)
- Quiz 2: Prompt Engineering & Context Engineering (48 MCQs, 2 hours)
- Module Project: Working AI agent system — n8n workflow + coded agent, deployed with Docker

MODULE 2 — Intelligence Layer: Agent SDKs, Memory, RAG, Design Patterns & Multi-Agent Systems

Duration	3 Months (Weeks 13–24)
Total Hours	~78 hours
Prerequisite	Module 1 completion

Module Goal

Master a production agent framework in depth with integrated short-term memory, ground agents in private knowledge using RAG, learn framework-agnostic agentic design patterns, build multi-agent systems, and implement long-term persistent memory.

Framework Track: One of the following three options will be selected before course launch. All subsequent module content (memory, RAG, design patterns, multi-agent systems) is taught using the chosen framework, but design patterns are framework-agnostic and transferable.Option A: OpenAI Agents SDKOption B: Google ADK (Agent Development Kit)Option C: LangChain v1 + LangGraph (latest versions)

Week 13–17 | Agent Framework Deep Dive + Short-Term Memory (5 Weeks)

Option A: OpenAI Agents SDK

Week 13: SDK architecture and philosophy, the four primitives (Agents, Tools, Handoffs, Runner), defining agent instructions and personas, @function_tool decorator, Runner.run_sync() vs streaming execution

Week 14: Agent cloning and dynamic instructions, structured tool inputs/outputs with Pydantic, error handling inside tool execution, built-in tools (web search, code interpreter), connecting agents to REST APIs

Week 15: Multi-agent orchestration: handoffs between specialized agents, supervisor and worker agent patterns, context objects and state passing between agents

Week 16: Short-term memory: conversation history, context window management, scratchpad patterns for intermediate reasoning, summarization memory for long conversations, Redis for fast session memory

Week 17: Session management across interactions, managing multiple concurrent user sessions, state persistence across agent turns. Practical project: Build a stateful multi-turn agent with tools and memory

Option B: Google ADK

Week 13: What ADK is: Google's open-source, model-agnostic framework, building agents with LlmAgent, tools, and orchestration, ADK's model flexibility via LiteLLM

Week 14: ADK's native MCP tool integration, building custom tools, Agent Garden: pre-built patterns and templates, bidirectional audio and video streaming

Week 15: Multi-agent orchestration with ADK: hierarchical structures and routing, agent-to-agent delegation, error handling and debugging

Week 16: Short-term memory: ADK session management, context window management, scratchpad and summarization patterns, Redis for fast session memory

Week 17: Managing concurrent sessions, state persistence. Practical project: Build a stateful multi-turn agent with tools and memory

Option C: LangChain v1 + LangGraph

Week 13: LangChain v1 — what changed: streamlined package, legacy moved to langchain-classic. create_agent — the single entry point for building agents. Model integrations, building tools as Python functions, structured outputs

Week 14: Middleware system: composable hooks for agent behavior. Built-in middleware: SummarizationMiddleware, HumanInTheLoopMiddleware, PII detection, FallbackMiddleware, ToolRetryMiddleware. Custom middleware with @before_agent, @after_agent decorators

Week 15: LangGraph — graph-based workflows: nodes, edges, state. Conditional routing, branching, loops, checkpointing. Deep Agents: prebuilt harness with planning tool, filesystem backend, sub-agent spawning. LangSmith integration

Week 16: Short-term memory: conversation history management, SummarizationMiddleware for context compression, LangGraph checkpointing for state persistence, Redis for session memory

Week 17: Session management, state persistence with LangGraph checkpointers. Practical project: Build a stateful multi-turn agent with tools and memory

Week 18–20 | RAG — Retrieval Augmented Generation

Vector Database: One of Qdrant or Pinecone will be selected before course launch.

Week 18: RAG Foundations

- What RAG is and why it outperforms fine-tuning for knowledge tasks
- Document loading: PDFs, web pages, databases, APIs
- Chunking strategies: fixed-size, recursive, semantic, document-aware
- Embedding models: what they are, how they work, choosing the right model
- Vector databases: Qdrant or Pinecone (one selected)

- Indexing and storing embeddings

Week 19: Retrieval Strategies & RAG Types

- Retrieval strategies: similarity search, hybrid search (keyword + semantic), re-ranking
- Naive RAG: basic retrieve + generate
- Advanced RAG: pre-retrieval optimization, post-retrieval re-ranking
- Modular RAG: composable retrieval pipelines
- Agentic RAG — agents that decide when, what, and how to retrieve
- Multi-hop RAG for complex multi-step question answering
- Graph RAG — knowledge graph enhanced retrieval
- Self-RAG — agents that self-reflect on retrieval quality

Week 20: Production RAG

- RAG evaluation: faithfulness, relevance, context recall, answer correctness
- Caching and optimization for production RAG
- Integrating RAG as a tool inside agents using the chosen framework
- Practical project: Build a RAG-powered agent over a document corpus

Week 21–23 | Agentic Design Patterns + Multi-Agent Systems

Week 21: Core Design Patterns (Framework-Agnostic)

- ReAct: Reasoning + Acting in a loop
- Plan-and-Execute: upfront planning then sequential execution
- Reflection: agents that critique and improve their own outputs
- Tool-use patterns: parallel tool calls, sequential tool chains
- Human-in-the-loop: escalation and approval workflows

Week 22: Orchestration Patterns

- Supervisor pattern: one orchestrator, many workers
- Hierarchical agents for complex task decomposition
- Swarm pattern: dynamic agent selection
- Pipeline pattern: sequential specialized agents
- Router pattern: intelligent routing to specialized agents
- When to use which pattern — decision framework
- Anti-patterns: common mistakes in agentic system design

Week 23: Multi-Agent Systems Implementation

- Implementing design patterns using the chosen framework
- Agent handoffs and routing strategies
- Context objects and state passing between agents
- Shared memory across agent teams
- Error handling and fallback strategies in multi-agent systems
- Combining patterns for complex workflows

Practical Project

Build a 3-agent pipeline (planner, researcher, writer) applying agentic design patterns with RAG integration and short-term memory.

Week 24 | Long-Term Memory

Memory Framework: One of mem0 or LangMem will be selected before course launch.

Option A: mem0

- What mem0 is: personalized memory layer for AI agents
- mem0 architecture and API
- Storing and retrieving user-specific memories

Option B: LangMem

- What LangMem is: LangChain's memory management framework
- LangMem architecture and API
- Storing and retrieving persistent memories

Common Topics (Both Options)

- Memory categories: episodic (what happened), semantic (what is known), procedural (how to act)
- Persistent storage with vector DBs and knowledge graphs
- Memory consolidation and forgetting strategies
- User-specific personalization memory
- Combining short-term (Weeks 16–17) and long-term memory in production agents
- PostgreSQL + pgvector for long-term semantic memory

Module 2 Assessment

- Quiz 3: Agent SDKs, RAG, Memory & Design Patterns (MCQ + code, 2 hours)
- Mid-Term Project: Knowledge-grounded agent — build an AI research assistant with short-term memory, long-term memory, RAG over a document corpus, and multi-agent orchestration

MODULE 3 — Collaboration Layer: FastAPI, Databases, Protocols, MCP, A2A, Evaluation & Autonomous Agents

Duration	3 Months (Weeks 25–36)
Total Hours	~78 hours
Prerequisite	Module 2 completion

Module Goal

Build production backends with FastAPI and Neon DB, understand protocol foundations, create custom MCP servers and A2A implementations, evaluate and observe agent systems, build fully autonomous agents, and deliver a complete end-to-end project.

Week 25–28 | FastAPI & Database Layer (Neon DB, SQLModel, Alembic)

Week 25: FastAPI Fundamentals

- What FastAPI is and why it's the choice for agentic backends
- Path operations, request/response models with Pydantic
- Dependency injection and middleware
- Async endpoints and background tasks
- OpenAPI/Swagger documentation (auto-generated)

Week 26: Database Layer

- Neon DB: serverless PostgreSQL — setup and configuration
- SQLModel: Python ORM that combines SQLAlchemy and Pydantic
- Defining models, relationships, and queries with SQLModel
- Alembic: database migrations — creating, running, and managing schema changes
- Connecting FastAPI to Neon DB with async SQLAlchemy

Week 27: Advanced FastAPI for Agents

- Streaming responses for LLM output (Server-Sent Events)
- Handling long-running agent tasks
- Basic JWT authentication for APIs
- Structuring FastAPI projects for agentic backends
- Background workers and task queues

Week 28: Building Agent APIs

- Designing RESTful APIs that serve AI agents
- Agent request/response patterns
- WebSocket connections for real-time agent communication
- Error handling and retry patterns for agent backends

- Practical project: Build a complete FastAPI backend with Neon DB that serves agent queries via streaming

Week 29 | Protocol Foundations

Networking & Protocol Basics

- HTTP fundamentals: request/response, methods, headers, status codes
- REST API basics: endpoints, CRUD operations, authentication
- JSON-RPC: what it is, how it differs from REST, when to use it
- Server-Sent Events (SSE) and streaming protocols
- Why these protocols matter for MCP and A2A

Week 30–31 | Custom MCP Servers & A2A Protocol

Week 30: Building Custom MCP Servers

- FastMCP library for rapid MCP server development
- Exposing tools, resources, and prompt templates via MCP
- MCP Security: OAuth 2.1 authentication
- Permission-based user approval for sensitive operations
- Connecting agents to custom MCP servers
- Tool discovery and dynamic capability loading
- MCP ecosystem: public registries and reusable server libraries
- Databases and RAG pipelines as MCP resources

Week 31: A2A Protocol

- What A2A is: standardized peer-to-peer agent communication
- A2A vs. direct function calls — when you need the protocol
- Agent Cards: advertising agent capabilities
- A2A message structure and task lifecycle
- The pattern: MCP for vertical tool integration, A2A for horizontal agent collaboration
- Writing and publishing Agent Cards
- Implementing A2A message handlers
- Authentication and security in A2A (OAuth 2.0-based)
- Google ADK and LangGraph A2A support
- Linux Foundation Agentic AI Foundation (AAIF) governance
- NANDA: identity, authorization, and audit layer for agents
- Practical project: Build agents communicating via custom MCP + A2A

Week 32 | Agent Evaluation, Testing, Tracing & Observability

Evaluation & Testing

- Manual evaluation frameworks and rubrics
- Automated evaluation with LangSmith
- RAG evaluation: RAGAS (faithfulness, relevance, context recall)

- Human-in-the-loop evaluation workflows
- Regression testing for agents: detecting behavior drift
- Cost tracking and latency benchmarking

LLM Economics: Token Pricing, Routing, Budgeting & Caching

- Token pricing across providers: understanding cost per input/output token
- Model routing: sending complex tasks to frontier models, simple tasks to cheaper models
- Budget management: setting cost limits, alerts, and spend tracking per agent/project
- Prompt caching: reducing redundant LLM calls for repeated/similar prompts
- Semantic caching: caching based on meaning, not exact matches
- Small Language Models (SLMs) as cost-effective alternatives for routine agent tasks
- Cost monitoring dashboards with LangSmith and Langfuse
- Optimizing agent pipelines for cost: reducing token consumption without losing quality

Tracing & Observability

- OpenTelemetry for distributed agent tracing
- LangSmith for LLM call tracing across agent networks
- Langfuse for open-source observability
- Logging agent decisions and tool calls
- Circuit breakers and retry policies

Week 33–35 | Fully Autonomous Agents

Week 33: Autonomous Agent Fundamentals

- What fully autonomous agents are: agents that operate continuously without human intervention
- Assisted vs. semi-autonomous vs. fully autonomous — the autonomy spectrum
- Use cases: browser-use agents, computer-use agents, continuous workflow automation
- The architecture of autonomous agents: perception, planning, action, memory loops
- Safety guardrails: bounded autonomy, escalation paths, human-in-the-loop checkpoints

Week 34: OpenClaw

- What OpenClaw is and its architecture
- Building autonomous agents with OpenClaw
- Browser automation and web interaction capabilities
- Task planning and self-correction mechanisms
- Integration with MCP and A2A protocols

Week 35: NanoClaw

- What NanoClaw is and how it differs from OpenClaw
- Lightweight autonomous agents for focused tasks
- Error recovery and graceful degradation
- Continuous monitoring and self-reporting
- Reversible actions and staged execution patterns

Week 36 | End-to-End Project

Module 3 Capstone

Build a fully autonomous agent system that independently researches a topic, gathers data from multiple sources via custom MCP servers, collaborates with other agents via A2A, stores results in Neon DB, and produces a structured report — with safety guardrails, observability via LangSmith, and human escalation for sensitive decisions.

Module 3 Assessment

- Quiz 4: FastAPI, MCP, A2A, Evaluation & Autonomous Agents (MCQ + code, 2 hours)
- Hackathon 1 (8 hours): Build a multi-agent system using custom MCP + A2A from a given problem brief. Teams of 2–3 students.
- End-to-End Project: Autonomous agent system with full backend, protocols, and observability

MODULE 4 — Production & Scale: Docker, Dapr, DACA, Ethics & Capstone

Duration	3 Months (Weeks 37–48)
Total Hours	~78 hours
Prerequisite	Module 3 completion

Module Goal

Move from working prototypes to production-grade, enterprise-ready systems. Master containerization with Docker, microservice architecture, the DACA (Dapr Agentic Cloud Ascent) pattern for scalable deployment, understand ethical responsibilities, and complete a capstone project demonstrating full-stack agentic AI engineering.

Week 37–39 | Docker & Microservices for Agents

Week 37: Docker Fundamentals

- What Docker is and why containerization matters for agents
- Writing production Dockerfiles for FastAPI + agent apps
- Docker images, containers, and registries
- Managing secrets and environment variables securely
- Multi-stage builds for smaller images

Week 38: Docker Compose & Multi-Service Development

- Docker Compose for local multi-service development
- Networking between containers
- Volume management and data persistence
- Running agent backends, databases, and MCP servers together

Week 39: Microservice Architecture for Agents

- Decomposing agentic systems into microservices
- Service communication patterns for agent backends
- API gateway patterns for agent services
- Structuring MCP servers as independent microservices
- Health checks and monitoring for containerized agents

Week 40–42 | Dapr & the DACA Pattern

Week 40: Dapr Building Blocks

- What Dapr is: distributed application runtime for microservices
- Service Invocation: reliable HTTP/gRPC between services
- State Management: agent state with Redis or CockroachDB

- Pub/Sub Messaging: event-driven agent communication with Kafka/RabbitMQ

Week 41: Advanced Dapr

- Workflows: durable, resumable multi-step agent processes
- Virtual Actors (Dapr Agents): stateful agents at massive scale
- Dapr sidecars and the sidecar pattern
- Dapr with Docker Compose

Week 42: The DACA Design Pattern

- AI-first + cloud-first design principles
- Stateless containerized agents with Dapr sidecars
- Scaling from local development to planetary scale
- Cost optimization with free-tier cloud + self-hosted LLMs
- The “10 million concurrent agents” architecture challenge

Production Infrastructure

- CockroachDB for globally distributed agent state
- RabbitMQ / Apache Kafka for high-throughput message queues
- Azure Container Apps (ACA) for serverless deployment
- ArgoCD for GitOps-based continuous deployment

Week 43–44 | Ethical AI, Safety & Governance

Week 43: AI Agent Ethics

- The governance imperative: why most agentic pilots fail (Deloitte 2025 data)
- Bounded autonomy: clear limits, escalation paths, and human-in-the-loop checkpoints
- Bias in LLM outputs and mitigation strategies
- Privacy and data protection in agentic systems

Week 44: Safety Architecture & Compliance

- Input/output guardrails: prompt injection defense, harmful content filtering
- Reversible actions and staged execution patterns
- Audit trails and explainability for agent decisions
- Circuit breakers to prevent cascading autonomous failures
- EU AI Act implications for autonomous agents
- NIST AI Risk Management Framework
- SOC 2 compliance for agent deployments
- Building trust through transparency and documentation

Week 45–48 | Capstone Project

Week 45: Project Proposal

- Define a real-world problem and an agent-driven solution
- Choose appropriate technologies from the full course stack

- Present architecture diagram and success metrics

Week 46–47: Design & Development

- Iterative development with weekly instructor check-ins
- Full agentic system: FastAPI backend, multi-agent pipeline, custom MCP servers, memory, agent orchestration, Dapr workflows
- Deployed with Docker Compose (local) or Dapr (production)

Week 48: Testing, Documentation & Presentation

- Rigorous testing and performance evaluation
- Technical documentation with architecture diagrams
- Final 20-minute presentation: demo, challenges, learnings, future roadmap

Suggested Capstone Ideas

- AI-powered research assistant: multi-agent pipeline with RAG, autonomous research, deployed with Docker
- Autonomous workflow engine: agents that process business forms, route tasks, notify stakeholders via A2A
- Personalized learning agent: adapts content based on student memory profile using mem0/LangMem
- Agentic DevOps assistant: autonomous agent that monitors infrastructure, diagnoses issues, proposes fixes

Module 4 Assessment

- Quiz 5: Docker + Dapr + DACA Pattern (simulation-based, 2 hours)
- Hackathon 2 (8 hours): Agent-native startup challenge — build an MVP of an agentic product
- Capstone Project (40% of final grade): Full system design, implementation, deployment, and presentation

Full Course Assessment Framework

Component	Weight	Description
Module Quizzes (5 total)	20%	MCQ + code analysis after each major topic
Hackathon 1 (MCP + A2A)	10%	8-hour team build challenge
Hackathon 2 (Agent Startup)	10%	8-hour agent product MVP
Mid-Term Project (Module 2)	20%	RAG + memory + multi-agent research assistant
Capstone Project (Module 4)	35%	Full system: design, code, deployment, presentation
Participation & Peer Review	5%	Active engagement, code reviews, peer feedback

Grading Scale

Grade	Score Range
A — Excellent	90–100%
B — Good	80–89%
C — Satisfactory	70–79%
D — Passing	60–69%
F — Failing	Below 60%

International Certification Pathway

Upon course completion, students are prepared for the following industry certifications:

1. **IBM RAG and Agentic AI Professional Certificate** — validates RAG, MCP, LangGraph skills
2. **NVIDIA Generative AI with LLMs Certification** — validates LLM fundamentals and generative AI practice
3. **AWS / Azure AI Engineer Certifications** — cloud deployment and managed AI services

What Makes This Curriculum 2026-Ready

- **MCP is now a core skill — it is the standard way agents access tools and data**

MCP has become the universal interface between AI agents and external systems.

- **A2A enables the multi-agent web — agents from different organizations can collaborate securely**

A2A protocol is now an open Linux Foundation standard enabling cross-organization agent workflows.

- **Multi-framework literacy — students learn transferable patterns across OpenAI SDK, Google ADK, and LangChain/LangGraph**

Framework-agnostic design patterns ensure students aren't locked into a single vendor.

- **Fully autonomous agents are the frontier — OpenClaw and NanoClaw represent cutting-edge agentic AI**

Students learn to build agents that operate independently with proper safety guardrails.

- **Governance and bounded autonomy are as important as technical skills**

Deloitte 2025 data confirms most agentic pilots fail due to governance gaps, not model quality.

- **DACA pattern addresses the hardest challenge: 10 million concurrent agents without failure**

The DACA architecture provides a blueprint for planetary-scale agentic system deployment.

Curriculum Version: March 2026 | Based on Panaversity learn-agentic-ai Repository & Advanced Python with AI Course. Updated with 2026 industry research from Deloitte, MachineLearningMastery, CIO.com, SoftmaxData, and Subhadip Mitra.